

Virtual Staining: Colorazione Istologica Virtuale

Andrea Mura

Università degli Studi di Cagliari

Corso di Laurea in Ingegneria Elettronica, Informatica e delle Telecomunicazioni

A.A. 2024/2025

Introduzione

Nel contesto dell'analisi istopatologica, è necessario colorare chimicamente i campioni per evidenziare le strutture cellulari e tissutali. Tuttavia, le tecniche di colorazione tradizionali, come l'ematossilina-eosina (H&E), richiedono tempo e risorse significative, rendendo il processo di diagnosi più lungo e complesso. Da questo problema è nato il progetto di *Virtual Staining*, con l'obiettivo di rendere meno laborioso e più rapido il processo di analisi istopatologica utilizzando la colorazione virtuale.

Il progetto è ispirato a studi precedenti, come quello presentato in [1], nel quale viene addestrata una rete neurale per trasformare immagini in autofluorescenza in immagini istologiche realistiche. Si è sviluppato un sistema capace di ottenere da un'immagine *label-free*, quindi non colorata, la sua controparte *stained*, riducendo i costi di produzione e i tempi di attesa necessari per permettere agli specialisti di effettuare l'analisi, permettendo inoltre il riuso della stessa immagine *label-free* e del campione per altre tecniche di colorazione, dato che non viene alterata irreversibilmente [2].

È stata progettata una pipeline che parte dall'**allineamento** tra immagini *label-free* e la controparte *stained*, procede con la **suddivisione in sottosezioni** e la creazione di un dataset strutturato, terminando con l'addestramento di una rete neurale **Pix2Pix** [3]. Questa architettura, composta da un **generatore** e da un **discriminatore**, permette alla rete neurale di apprendere come trasformare un'immagine dal dominio originale *label-free* a un dominio *stained* attraverso un approccio supervisionato. Si ottiene così un sistema in grado di generare immagini colorate in modo realistico a partire da immagini non colorate e inalterate, con importanti applicazioni in ambito diagnostico [4]. Una conseguenza di questo metodo è la possibilità di combinare varie tecniche di colorazione in un'unica immagine, mettendo in risalto vari componenti che con metodi tradizionali sarebbe impossibile ottenere, permettendo un'analisi più rapida e accurata [5].

1 Fasi del lavoro svolto

1. **Coregistrazione delle immagini:** dato che le scansioni originali *label-free* e *stained* presentavano un inevitabile disallineamento, con l'ausilio di **OpenCV** [6] e degli algoritmi *SIFT* e *ORB* è stata calcolata la matrice di trasformazione attraverso il rilevamento di punti distintivi dell'immagine. Dopo vari test, è stato utilizzato l'algoritmo *SIFT* per l'estrazione dei *keypoints* da un'immagine normalizzata tramite *CLAHE*, con l'ausilio di maschere e di filtri di distanza per estrarre corrispondenze più forti. La matrice di trasformazione calcolata è stata applicata all'intera immagine *stained* e alla relativa maschera, ottenendo così un'immagine allineata pixel per pixel con la corrispondente immagine *label-free*.
2. **Suddivisione in sottosezioni:** dopo aver ottenuto l'immagine allineata, si è proceduto a suddividerla in sottosezioni da 512×512 pixel, utilizzando una griglia con passo di 300 pixel. L'utilizzo di questa tecnica, che causa sovrapposizione tra sezioni, ha permesso di generare un numero di coppie *label-free/stained* sufficiente per l'addestramento della rete neurale. Per garantire la qualità del dataset, sono state filtrate le sottosezioni che contenevano prevalentemente sfondo, escludendo quindi le regioni non informative.
3. **Creazione del dataset strutturato:** le sottosezioni ottenute sono state suddivise casualmente in un dataset strutturato secondo la convenzione **train/val/test**, con percentuali rispettivamente del 70%, 15% e 15%. Per garantire il determinismo della partizione è stato utilizzato un seme randomico fissato.

4. **Addestramento della rete Pix2Pix:** la rete neurale è stata implementata sul framework PyTorch [7] sfruttando l'architettura *Pix2Pix*, formata da un generatore *UNet* e da un discriminatore *PatchGAN*. È stata utilizzata una combinazione di funzioni di perdita *Binary Cross Entropy* e *Mean Absolute Error* (L1), come proposto in [3], per influenzare l'apprendimento del generatore e del discriminatore. Alla fine di ogni epoca di addestramento è stata eseguita una validazione per monitorare l'andamento delle prestazioni sui dati di *validation*, permettendo così di verificare se stesse avvenendo overfitting.

2 Risultati ottenuti

La fase di testing ha permesso di confermare l'efficacia del modello addestrato, ottenendo immagini colorate di output che, nella maggior parte dei casi, presentano le strutture e le colorazioni delle rispettive immagini *stained* reali. La fedeltà tra immagini è stata valutata principalmente attraverso un confronto diretto tra triple formate da immagine *label-free*, immagine colorata virtualmente e immagine *stained* reale, generate automaticamente.

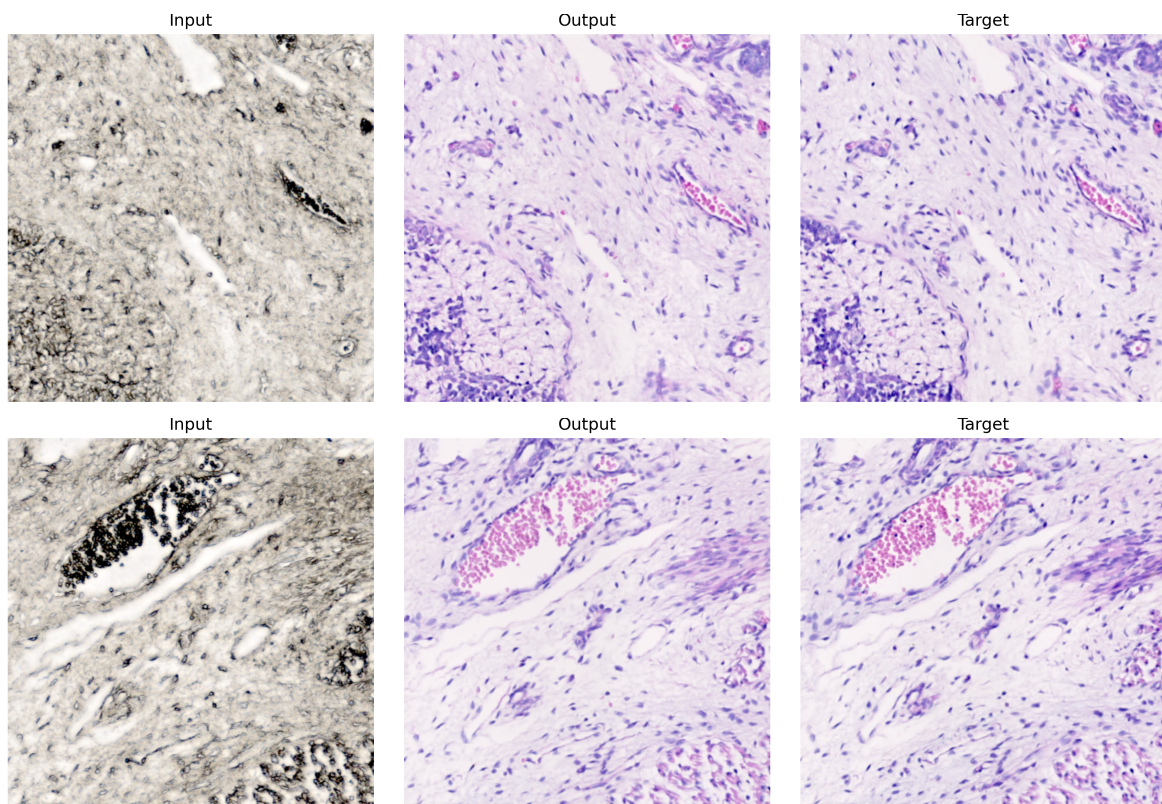


Figura 1: Esempi di triple di immagini: *label-free*, colorazione virtuale e immagine *stained* reale.

Due esempi sono mostrati in Figura 1, dove si può notare come l'immagine generata sia molto simile a quella reale, con una buona corrispondenza delle strutture e dei colori.

3 Competenze acquisite

Durante lo svolgimento del progetto sono state acquisite competenze tecniche e organizzative sia nell'ambito del processamento delle immagini e del deep learning, sia nella suddivisione del lavoro e nel brainstorming. In particolare, sono state apprese tecniche di *preprocessing* e *allineamento*, manipolazione di immagini tramite matrici, e l'uso di algoritmi di deep learning per la generazione di immagini, come il framework PyTorch.

È stato possibile osservare il rapporto causa-effetto tra i modelli e i parametri delle reti neurali e le immagini generate. Sono state sperimentate varianti più avanzate del modello, come l'uso di *UNet* e di un discriminatore migliorato, simile a quanto proposto in [8, 9]. Inoltre, per la gestione dei tempi, si

è compreso come l'impostazione dei batch su GPU, l'impiego di tecnologia NVIDIA come *AMP*, l'ottimizzazione di più funzioni di perdita, e il salvataggio dei checkpoint abbiano permesso un monitoraggio efficace dell'apprendimento.

Infine sono state acquisite competenze trasversali che hanno favorito una migliore organizzazione, come la gestione del logging sia per l'allineamento delle immagini che per l'addestramento della rete neurale, la validazione periodica e la generazione automatica dei risultati grafici. Queste abilità hanno contribuito ad affinare le capacità di documentazione e di valutazione del comportamento della pipeline.

4 Considerazioni finali

Le sfide affrontate durante il progetto hanno permesso di applicare concetti teorici a un problema reale, affrontando questioni pratiche legate alla qualità dei dati in origine, alla progettazione di modelli di deep learning e, in generale, alla ricerca su un argomento innovativo.

Riteniamo che questo progetto, nella sua interezza, possa rappresentare un punto di partenza per chi voglia approfondire il tema della **Virtual Staining**. La presenza di notebook e script modulari rende la pipeline facilmente adattabile a modifiche e a nuovi dataset. Il codice è reso disponibile pubblicamente nella repository GitHub: <https://github.com/RikiXY/Virtual-Staining>, con l'intento di favorire la condivisione dei risultati e contributi futuri.

Riferimenti bibliografici

- [1] Y. Rivenson *et al.*, “Virtual histological staining of unlabelled tissue-autofluorescence images via deep learning,” *Nature Biomedical Engineering*, vol. 3, n. 6, pp. 466–477, 2019. doi: 10.1038/s41551-019-0362-y.
- [2] L. Latonen, S. Koivukoski, U. Khan e P. Ruusuvuori, “Virtual staining for histology by deep learning,” *Trends in Biotechnology*, vol. 42, n. 9, pp. 1177–1186, 2024. doi: 10.1016/j.tibtech.2024.02.009.
- [3] P. Isola, J.-Y. Zhu, T. Zhou e A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, pp. 1125–1134, 2017. doi: 10.1109/CVPR.2017.632.
- [4] W. Lin, Y. Hu, R. Zhu, B. Wang e L. Wang, “Virtual staining for pathology: Challenges, limitations and perspectives,” *Intelligent Oncology*, vol. 1, n. 1, pp. 105–119, 2025. doi: 10.1016/j.intonc.2025.03.005.
- [5] M. Kawai *et al.*, “Virtual multi-staining in a single-section view for renal pathology using generative adversarial networks,” *Computers in Biology and Medicine*, vol. 182, p. 109149, 2024. doi: 10.1016/j.compbimed.2024.109149.
- [6] OpenCV Developers, “OpenCV: Open Source Computer Vision Library,” <https://opencv.org>.
- [7] PyTorch Developers, “PyTorch: Open source machine learning framework,” <https://pytorch.org>.
- [8] B. Bai *et al.*, “Deep learning-enabled virtual histological staining of biological samples,” *Light: Science & Applications*, vol. 12, n. 57, pp. 1–20, 2023. doi: 10.1038/s41377-023-01104-7.
- [9] U. Khan *et al.*, “The effect of neural network architecture on virtual H&E staining: Systematic assessment of histological feasibility,” *Patterns*, vol. 4, p. 100725, 2023. doi: 10.1016/j.patter.2023.100725.